ناطق · NAATIQ — DOCUMENTATION

Building with AI

The build-session prompts, the working method, and an honest account of what went wrong

1. What this document is

Naatiq was built end to end in a single collaborative session between a human (Karam) and Claude running in Cowork with tool access to Supabase, GitHub, the shell and the web.

This document records the **build-session prompts** — the instructions that produced the codebase. They are distinct from the **product system prompts** in document 05, which run in production every time a worker sends a message.

2. The prompts that shaped the build, in order

Reproduced verbatim, lightly grouped. This is a selection chosen to carry the narrative — **document 12 holds the complete record**, every prompt in order with what each one produced.

Framing and guardrails

> *"please read the md file and tell me what you will do, if you have any questions please state them before implementing. do not do anything extra or connect to any app without running it by me first."*

The most consequential prompt of the session. It established two rules that held throughout: **plan before implementing**, and **no unrequested side effects**. Every external connection — Supabase, GitHub, Railway, Twilio — was proposed and approved before it was made.

> *"also github repo is https://github.com/Kjudeh/Kanz-Hackathon.git please connect to it and have everything pushed there"*

Direction and scope, via structured choices

Rather than long specifications, most scope decisions were made through short multiple-choice questions. The answers, in order, were: *Full end-to-end build · Nothing yet — greenfield · Yes, read & write · New project · Classic GitHub token · Build Phase 1 + 2 code now · Build Phase 3: the CV · Build Phase 4: the dashboard · Arabic voice replies · Agent + CV + docs · Deploy Gotenberg · Cut voice replies for now · Full CV access · Both, as separate docs · Separate PDF per document.*

This kept the human in control of direction while delegating implementation, and it is why the build never stalled waiting for a long written spec.

Brand and identity

> *"i added a file 2026-07-18 13.54.40.jpg please use that as the logo for the dashboard, and use the colours in it for as a moodboard for the dashboard, we also changes the name of the app to ناطق"*

A single image drove the entire visual system. The palette was sampled directly from the logo — navy `#001737`, blue `#0E52E5`, emerald `#09A385` — and then applied consistently to the dashboard, the CV template, the landing page, the employer portal and these documents.

> *"i need the dashboard in arabic with english as an option" "can you add the logo to the footer of the CV?"*

Getting it live

> *"perfect now what api tokens and secrets do you want from me?" "these are added. do we need gotenberg? on my beautistaff, we generate the pdfs without a third party tool" "everything is set; all the keys and twilio webhook" "done, please configure, and clean the database from the all the candidates and demo ones, so i can start testing."*

The Gotenberg question was a good challenge and got a real answer rather than a defence: PDFs can be generated without a third-party service, but **Arabic** PDFs are a different problem. Arabic needs contextual letter shaping and bidirectional layout, which most PDF libraries get wrong. Gotenberg wraps headless Chromium, so it inherits the browser's text engine. It is self-hosted, not a vendor — the reasoning is in document 07.

Debugging, from the person actually using it

> *"sent a voice message im receiving nothing, please fix" "perfect now waiting for the CV to be sent, i am receiving correct replies and its understanding me correctly, but i am not receiving replies as voice notes only written ones." "on the dashboard it shows that i have completed the conversation but no link to the pdf" "if the dahsboard is live why is it showing demo data?"*

Every one of these was a real bug found by a real person using the product on a real phone. None would have surfaced from reading the code.

Widening the surface

> *"i need you to create a landing page with the whatsapp number... a call to action button, how many CVs generated, a page for clients to find the CVs and employ, how many employments have happened. who we are targeting which candidates, which jobs."*

This prompt caused a schema change. There was no honest way to display an employment count, so a `placements` table was added rather than inventing a number.

> *"now the most important part i need you to create a diagram that shows the whole process" "create a documentation file, and create pdfs of all the prompts used, tools used, an updated prd, target audience, technical document, and see what else missing from documentation that we need to add."*

3. The working method

Plan first, then build. The opening constraint set the pattern for the whole session. Every phase began with a stated plan and open questions.

Structured questions instead of long specs. Direction was set through short multiple-choice decisions, which is far faster than writing a specification and produces fewer misunderstandings.

Build the risky thing first. Arabic PDF rendering was identified as the highest-risk component and was solved early with a locally rendered sample — not left to the final night, which is when this class of problem usually surfaces.

Test with a real person on a real phone. Every significant bug came from actual use.

Make the system explain itself. The single highest-leverage decision of the build; see below.

Own mistakes plainly. When a recommendation was wrong, it was named as wrong and corrected.

4. The turning point: making the system explain itself

Supabase's log API exposes request lines but not `console.error` output. For several hours, debugging meant guessing at a black box: a voice note went in, nothing came out, and there was no way to see why.

The fix was to stop guessing and build a **secret-gated self-test endpoint** into the webhook. It reports every environment variable's presence and the resolved configuration, and on demand round-trips a message through Claude, sends a live WhatsApp message, exercises the TTS path, or triggers CV generation for a given user.

It paid for itself immediately. **Four separate production bugs** were each identified in minutes rather than hours: the rejected `temperature` parameter, the Twilio channel misconfiguration, the missing URL scheme on the Gotenberg host, and the ElevenLabs plan restriction. The same philosophy was then applied to `generate-cv` via a `debug: true` mode that returns a `steps[]` trace.

The general lesson: **when a system is hard to observe, invest in observability before investing in more guesses.**

5. What went wrong

Recorded honestly, because a build log that contains no mistakes is not a build log.

ElevenLabs voice IDs — wrong twice. Arabic voice replies returned HTTP 402. The recommended fix was a specific premade voice ID; it failed again. A second voice ID was recommended; it failed again. Only then was the actual cause established by searching the documentation: **the ElevenLabs voice library is not available over the API on free plans** — no voice ID would have worked. Two cycles were spent pattern-matching on an error code instead of reading the docs. The error was owned explicitly at the time, and the feature was shipped behind a flag rather than abandoned.

A stale model identifier. The code initially referenced an outdated Claude model string. Fixed by checking current documentation rather than relying on memory — a reminder that model names are present-day facts, not knowledge.

Git broke against the mounted folder. The user's folder could not unlink files (`EPERM`), which left git unusable. Worked around by mirroring the repository to a temporary directory and pushing from there.

Two GitHub tokens rejected. Fine-grained personal access tokens returned 403 twice. Resolved by switching to a classic token with `repo` scope.

Several deploys rejected. A stray `